Assignment - 9

K-Medoids Clustering: PAM, CLARA and CLARANS

Objective

Implement and compare K-Medoids clustering and its scalable variants—PAM (Partitioning Around Medoids), CLARA (Clustering Large Applications), and CLARANS (Clustering Large Applications based upon RANdomized Search). Understand how representative observations called medoids are selected to form clusters.

Dataset Preparation → Distance Calculation → Medoid Selection → Point Assignment → Objective-Cost Calculation → Search/Refinement → Evaluation

Implement the core algorithms using Python and NumPy. Do not use a ready-made K-Medoids, CLARA, or CLARANS implementation for the main experiments.

Software Requirements

• Python 3.x; NumPy and pandas; Matplotlib; scikit-learn for loading Iris and optional evaluation metrics; Jupyter Notebook / Google Colab / Python IDE.

Instructions

• Complete all tasks in one notebook or Python file.

• Use sklearn.datasets.load_iris() only to load the dataset; do not use ready-made clustering implementations for the main algorithms.

• Use fixed random seeds where applicable and report algorithm parameters.

• Show intermediate distance calculations, medoid choices, assignments, objective costs, and algorithm comparisons.

• Include tables, plots, observations, and a conclusion.

Part A: Dataset and Distance Preparation

Dataset: Iris from sklearn.datasets. Use the four numerical measurements for clustering. Keep species labels separate and use them only for post-clustering evaluation, never as model input.

1. Load Iris and display the number of records, numerical features, feature names, and class names.

2. Create feature matrix X from the four numerical attributes and retain species labels separately.

3. Display the first 10 records and check for missing or infinite values.

4. Standardize numerical features with StandardScaler for the main experiment; retain original values for an optional comparison.

5. Explain why class labels must not be used to construct clusters in unsupervised learning.

Advanced Data Analytics Lab - K-Medoids, CLARA and CLARANS

Task 1: Inspect the Dataset

1. Print record count and feature count.
2. Display feature names, class names, and class counts.
3. Show descriptive statistics for each feature.
4. Plot at least two feature-pair scatter plots, optionally coloured by species for visual inspection.
5. Explain one strength and one limitation of Iris as a clustering dataset.

Task 2: Implement Distance Calculation

For observations x and y, implement Euclidean distance:

$$d(x,y) = \text{sqrt}(\text{sum over features } j \text{ of } (x_j - y_j)^2)$$

1. Write euclidean_distance(x, y).
2. Test it on at least three pairs of observations and display distances.
3. Optionally implement Manhattan distance and compare results.
4. Explain how feature scaling affects distance-based clustering.
5. Use one consistent distance metric for the main comparison.

Task 3: Construct and Inspect a Distance Matrix

1. Compute the pairwise distance matrix for a subset of at least 10 observations.
2. Display the matrix with row and column indices.
3. Verify that diagonal entries are zero and the matrix is symmetric.
4. Explain why storing an N x N matrix can be expensive for large datasets.
5. State whether the main implementation stores distances or calculates them on demand.

Task 4: Define the K-Medoids Objective

For medoids M = {m1, ..., mk}, assign each point to its nearest medoid and minimize:

$$\text{Cost}(M) = \text{sum_i min over medoids } m_j \text{ of } d(x_i, m_j)$$

1. Write assign_to_nearest_medoid(X, medoids), returning assignments and distances.
2. Write total_cost(X, medoids), returning the sum of distances to the nearest medoid.
3. Test both functions on a small hand-selected example with two medoids.
4. Explain why a medoid is an actual observation, unlike a centroid.
5. Use k = 3 for the main Iris experiments and justify this choice.

Part B: K-Medoids using PAM

PAM starts with k medoids, assigns observations to their nearest medoids, and searches for medoid/non-medoid swaps that reduce total clustering cost.

Task 5: Initialize Medoids

1. Write a function to select k distinct observations as initial medoids.
2. Use a fixed random seed and print indices and feature values.
3. Calculate and display initial objective cost.
4. Repeat initialization with at least three seeds and record initial costs.
5. Explain why different starting medoids may produce different local solutions.

Advanced Data Analytics Lab - K-Medoids, CLARA and CLARANS

Task 6: Implement the PAM Swap Step

1. Identify current medoids and candidate non-medoid observations.
2. Create candidate solutions by replacing one medoid with one non-medoid.
3. Reassign points to nearest medoids and calculate candidate cost.
4. Accept a swap only if it reduces objective cost.
5. Print outgoing medoid, incoming observation, old cost, new cost, and acceptance decision.

Task 7: Complete PAM

1. Implement pam(X, k, max_iter or equivalent stopping condition).
2. Repeat swap evaluation until no improving swap exists or the iteration limit is reached.
3. Return medoid indices, assignments, total cost, and iteration/swap information.
4. Print cost after each accepted swap and verify that accepted swaps do not increase cost.
5. Run PAM with k = 3 and display final medoids and cluster sizes.

Task 8: Examine PAM Results

1. Create a table of medoid index, medoid feature values, cluster size, and within-cluster distance summary.
2. Plot clusters using two features and mark medoids distinctly.
3. Calculate total objective cost and average distance to assigned medoid.
4. Calculate ARI and/or NMI against species labels only after fitting; labels must not influence clustering.
5. Write two observations about cluster quality and limitations.

Part C: CLARA — Clustering Large Applications

CLARA draws samples, applies a medoid method to each sample, and evaluates each candidate medoid set on the full dataset. State sample size and number of samples explicitly.

Task 9: Implement CLARA Sampling

1. Choose and report a sample size appropriate for the dataset.
2. Write a reproducible function to draw a sample without replacement.
3. Ensure the sample contains at least k observations.
4. Display sample indices and compare sample feature distributions with the full dataset.
5. Explain why sampling saves computation but may miss good representative observations.

Task 10: Implement CLARA

1. Implement clara(X, k, sample_size, num_samples, ...).
2. For each sample, run your PAM implementation on the sampled data.
3. Map sample medoids back to original dataset indices.
4. Assign every point in the full dataset to its nearest candidate medoid and calculate full-data cost.
5. Keep the candidate medoids with the lowest full-data cost across samples.
6. Return final medoids, assignments, cluster sizes, best cost, and per-sample costs.

Advanced Data Analytics Lab - K-Medoids, CLARA and CLARANS

Task 11: Analyze CLARA

1. Run CLARA with k = 3 and at least five samples.
2. Print each sample's cost evaluated on the full dataset and identify the best sample.
3. Display final medoids and cluster sizes.
4. Compare CLARA cost with PAM cost from Task 7.
5. Explain CLARA's efficiency advantage and the risk introduced by sampling.

Part D: CLARANS — Randomized Local Search

CLARANS explores a randomized neighbourhood of possible medoid sets. A neighbour replaces one current medoid with a randomly selected non-medoid. The algorithm accepts improving neighbours and uses configured neighbour limits and restarts.

Task 12: Define a CLARANS Neighbour

1. Write a function to generate one valid neighbour by randomly choosing a medoid and a non-medoid observation.
2. Ensure that the candidate contains k distinct observations.
3. Calculate candidate cost and the cost difference from the current solution.
4. Show at least five neighbour proposals and whether each improves cost.
5. Explain random neighbour proposals versus testing every possible swap.

Task 13: Implement CLARANS

1. Implement clarans(X, k, num_local, max_neighbors, random_state); define each parameter.
2. Start from a valid set of k medoids.
3. At each step, sample a neighbour and accept it if its cost is lower.
4. Count unsuccessful neighbour checks and stop a local search when its configured limit is reached.
5. Perform the configured number of restarts and retain the best solution.
6. Return best medoids, assignments, cost, restart information, and cost history.

Task 14: Analyze CLARANS Search Behaviour

1. Run CLARANS with k = 3 and a fixed seed.
2. Record neighbour proposals, accepted swaps, unsuccessful checks, and restarts.
3. Plot objective cost against accepted swaps or search steps.
4. Run at least three seeds and report final costs.
5. Explain how randomized exploration affects computation and why results vary.

Advanced Data Analytics Lab - K-Medoids, CLARA and CLARANS

Part E: Evaluation and Comparison

Task 15: Compare PAM, CLARA and CLARANS

1. Run all three algorithms on the same preprocessed data with k = 3 and the same distance metric.
2. Document PAM stopping rules, CLARA sample size/number of samples, and CLARANS neighbour/restart limits.
3. Create a table with algorithm, objective cost, cluster count, runtime, and parameters.
4. Where possible, count distance evaluations or swap/neighbour evaluations as a hardware-independent measure of work.
5. Explain why fair runtime comparisons require the same machine, data, preprocessing, and stopping conditions.

Task 16: Cluster Quality Evaluation

1. Calculate silhouette score using the chosen distance metric where supported.
2. Calculate ARI and/or NMI with species labels only for post-hoc evaluation.
3. Compare algorithms in a table and identify the best score for each metric.
4. Explain why lower objective cost does not necessarily mean better agreement with species labels.
5. Identify one limitation of each evaluation metric.

Task 17: Effect of Number of Clusters (k)

1. Repeat experiments for k = 2, 3, 4, and 5.
2. Record objective cost, silhouette score, and cluster sizes for each algorithm where practical.
3. Plot k versus objective cost and k versus silhouette score.
4. Explain why optimal objective cost generally cannot increase as k increases, while silhouette score may rise or fall.
5. Discuss how the preferred k depends on the objective being optimized.

Task 18: Effect of CLARA Sample Size

1. Run CLARA with at least three valid sample sizes.
2. Keep number of samples and random-seed policy documented.
3. Record full-data cost and runtime for each sample size.
4. Plot sample size versus objective cost and runtime.
5. Explain the trade-off between efficiency and the chance of missing good medoids.

Advanced Data Analytics Lab - K-Medoids, CLARA and CLARANS

Task 19: Effect of CLARANS Search Parameters

1. Run CLARANS with at least three settings for max_neighbors and/or num_local.
2. Record objective cost, runtime, neighbour proposals, and accepted swaps.
3. Plot search effort versus final objective cost.
4. Explain how larger search budgets may improve solutions but increase computation.
5. Use fixed seeds and discuss variation across repeated runs.

Task 20: Required Visualizations

1. Create a cluster scatter plot for each algorithm using the same two features and mark medoids.
2. Create a bar chart comparing objective costs of PAM, CLARA, and CLARANS.
3. Compare runtime or distance-evaluation counts across algorithms.
4. Plot k versus objective cost and k versus silhouette score.
5. Plot CLARA sample size versus cost/runtime and CLARANS search effort versus cost.
6. Label axes, add titles and legends, and write an observation below each plot.

Task 21: Complexity and Practical Observations

1. Discuss the cost of pairwise distances and evaluating medoid swaps.
2. Explain why PAM can become expensive as dataset size grows.
3. Explain how CLARA limits medoid search to samples but evaluates candidates on the full dataset.
4. Explain how CLARANS uses randomized neighbourhood search rather than exhaustive neighbourhood exploration.
5. State one advantage, limitation, and suitable use case for each method.

Task 22: Final Results Table

Prepare a compact final summary table:

Experiment	Setting	Result to Report
PAM	k = 3	Medoids; cost; runtime; cluster sizes
CLARA	k = 3; stated sample settings	Best full-data cost; best sample; runtime
CLARANS	k = 3; stated search settings	Best cost; proposals; accepted swaps; runtime
Evaluation	Same preprocessing and k	Silhouette; ARI/NMI
k experiment	k = 2, 3, 4, 5	Cost and quality metrics
Parameter experiments	CLARA sample size; CLARANS search budget	Cost/runtime trade-offs

Advanced Data Analytics Lab - K-Medoids, CLARA and CLARANS

Task 23: Observations and Conclusion

1. Write 4–6 observations about clusters and selected medoids.

2. Write 2–3 observations comparing objective cost, runtime, and search effort across PAM, CLARA, and CLARANS.

3. Explain the trade-off between exhaustive local search, sampling, and randomized neighbourhood search.

4. Discuss the impact of k, scaling, sample size, and search parameters.

5. Conclude with applications such as customer segmentation, robust grouping, and clustering with non-Euclidean dissimilarities.

Submission Checklist

- Single Jupyter Notebook or Python source file.
- Dataset loading, preprocessing, and distance calculations.
- Distance-matrix inspection and objective-cost function.
- From-scratch PAM implementation and swap outputs.
- From-scratch CLARA implementation with sample and full-data costs.
- From-scratch CLARANS implementation with randomized neighbour search and restart tracking.
- Medoid, assignment, cluster-size, and objective-cost tables.
- Evaluation metrics, required plots, parameter experiments, observations, and conclusion.

Important Notes

- Do not use ready-made K-Medoids, CLARA, or CLARANS implementations for the main solution. scikit-learn may be used for loading, preprocessing, plotting support, and evaluation metrics.
- Use the same feature preprocessing and distance metric for fair comparisons.
- Medoid indices must refer to actual observations in the original dataset.
- Report objective cost consistently as the sum of distances to nearest medoids; if using squared distances or another objective, state it explicitly.
- Actual runtime and results depend on initialization, random seed, implementation, and parameters; report measured outputs rather than inventing expected values.

Reference Topics

K-Medoids and PAM swaps; CLARA sampling and full-data evaluation; CLARANS randomized neighbourhood search; distance-based clustering; objective cost; silhouette score; and external evaluation with ARI/NMI.